

ЛАБОРАТОРНЫЕ РАБОТЫ

Web-технологии

Версия 2026 · базовый и ИИ-трек

ФИОЛЕТОВЫЙ БЛОК — ТРЕБОВАНИЯ ИИ-ТРЕКА

Выполняется студентами, выбравшими разработку с ИИ и кодирующими агентами, в дополнение к основной части. Студенты базового трека пропускают эти блоки без потери баллов. Добровольные расширения отмечены отдельно.

Общий принцип

Студент отвечает за постановку задачи, архитектуру, ограничения и приемку результата. В ИИ-треке агент участвует в создании основных артефактов на каждом этапе: от требований и спецификации до кода, инфраструктуры, интерфейса и проверок. Результат принимается по фактам: работающей системе, корректным контрактам, тестам и измерениям.

Редакция 08.09.2026 · усиленный AI-driven процесс

Содержание

Лабораторная работа	ИИ создает и проверяет
COMMON	
WebLab#1 — I'm a startuper (and/or a system architect)	Требования, архитектура и контекст агента
WebLab#2 — REST API System Analyst	OpenAPI и проверка контракта
WebLab#3 — REST API Developer	Сервер по контракту и тесты
WebLab#4 — WEB SERVER Junior	Конфигурация развертывания и маршрутизации
BACKEND	
WebLab#5 — WEB SERVER Senior DevOps	Нагрузочный эксперимент и улучшение стенда
WebLab#6 — Backender	Декомпозиция и реализация сервисов
FRONTEND	
WebLab#7 — Designer	Дизайн-система и макеты
WebLab#8 — Frontender	Генерация клиента и реализация UI
WebLab#9 — Frontender PRO MAX	Расширения браузера и оценка их эффекта

Единые правила ИИ-трека

- Студент задает цель, ограничения и критерии приемки; ИИ создает основные артефакты этапа. Конкретная модель не фиксируется. Использование агента совместимо с индивидуальной сдачей при личной защите результата; правила баллов и бонусов сохраняются.
- Рабочий цикл: задание с входными файлами и критериями → план → генерация/изменение → запуск проверок → ревью → исправление. Для каждой лабораторной показать хотя бы один завершённый цикл; описать способ проверки, если исправления не потребовались.
- Вести один AGENTS.md или AI_CONTEXT.md со ссылками на актуальные требования, контракты и ADR, командами запуска/проверки и границами изменений. Перед новой задачей обновлять контекст. Секреты и реальные персональные данные в задания агенту не включать.
- В AI_REVIEW.md достаточно короткой записи на лабораторную: задача, инструмент/модель, ссылки на результат и проверки, существенное принятое или отклоненное решение с причиной. Полные чаты, количество промптов и процент сгенерированного кода не оцениваются; ошибки ИИ выдумывать не требуется.
- Ожидаемые результаты проверок выводить из требований и контрактов. Утверждение ИИ подтверждать запуском, измерением, документацией или анализом артефакта. При изменении требований сначала согласованно обновлять исходный контракт, затем зависящие от него материалы и код.
- На защите объяснить существенные решения и любой предъявленный фрагмент, показать реальные проверки и выполнить небольшое изменение с ИИ. Для передачи между лабораторными фиксировать версию результатов в репозитории; документы и отчеты можно хранить разделами уже существующих файлов.
- ИИ-трек определяет способ разработки. ИИ-функция внутри продукта — отдельное добровольное расширение WebLab#9; она не подменяет цели других лабораторных.

WebLab#1

I'm a startuper (and/or a system architect)

Depends on: [нет](#)

Subject: Анализ/проектирование

Season: [2026](#)

1. Основа лабораторных работ по Web - лабораторные работы по ППО. Минимальный стартовый набор - 6 лабораторных работ. Рекомендуемый - 7+.
2. Лабораторные работы разбиты на 3 группы:
 - a. Common: WebLab#1 - WebLab#4
 - b. Backend: WebLab#5 - WebLab#6
 - c. Frontend WebLab#7- WebLab#9
3. При выполнении ЛР самостоятельно и в срок, можно получить 100% баллов за ЛР. При выполнении ЛР в команде (2 человека) можно получить по 50% баллов на каждого члена команды. При выполнении ЛР самостоятельно не в срок можно получить 50% баллов. При выполнении ЛР в команде не в срок можно получить, соответственно, по 25% баллов на каждого члена команды.
4. У лабораторных работ есть допы, - это доп задания стоимостью 1 балл. Допы являются атомарными и никогда не делятся. Т.е. и в случае просрочки сдачи, и в случае парной работы доп гарантированно дает 1 балл каждому члену команды.
5. Баллы распределены таким образом, чтобы можно было получить зачет только благодаря сдаче ЛР:
 - a. Fullstack (solo): WebLab#1 - WebLab#5 + WebLab#7 - WebLab#8
 - b. Backend (solo): WebLab#1 - WebLab#6
 - c. Frontend (solo): WebLab#1 - WebLab#4 + WebLab#7 - WebLab#9
 - d. Dreamteam (team): WebLab#1 - WebLab#9 + Dops
6. По согласованию с преподавателем допускается уточнение идеи/цели/предметной области проекта из РРО.
7. В .md файле подготовить (с переиспользованием существующих материалов), кратко:
 - a. Цель работы, решаемая проблема/предоставляемая возможность
 - b. Краткий перечень функциональных требований и ключевых нефункциональных требований: как минимум по одному измеримому требованию к производительности, надежности и безопасности.
 - c. Use-case диаграмма системы
 - d. BPMN диаграмма основных бизнес-процессов (не менее 3, нелинейные)
 - e. Примеры описания основных пользовательских сценариев (не менее 3)
 - f. ER-диаграмма сущностей
 - g. Технологический стек (язык+фреймворки бекенда, язык+фреймворки фронтенда, БД, механизм доставки приложения)
 - h. Диаграмма БД
 - i. Контекстная, контейнерная и компонентная диаграммы системы (C4 Context/Container/Component).
 - j. Экраны будущего web-приложения на уровне черновых эскизов. Задача данного упражнения - понять, как с приложением должен взаимодействовать пользователь и

упростить последующее проектирование API. Это могут быть классические wireframes, черновики от руки, наброски в PAINT/псевдографике/Figma.

- к. Краткие записи об архитектурных решениях (не менее 2 ADR): выбранное решение, рассмотренные альтернативы и причины выбора.

Примечание: здесь не требуется полноценный UI/UX дизайн, требуется примерное распределение функциональности по экранам и возможным компонентам управления.

ИИ-ТРЕК · WebLab#1

Обязательные требования для выбравших ИИ-трек.

Вход: Материалы и репозиторий ППО, идея продукта и ограничения из основного задания.

- **Разобрать исходный проект.** Поручить ИИ изучить материалы ППО и составить карту существующих компонентов, ограничений и пробелов в требованиях. Ответить на существенные вопросы; отделить подтвержденные факты от предположений.
- **Сформировать задание агенту.** Создать с помощью ИИ AGENTS.md или AI_CONTEXT.md: цель, словарь, границы системы, ссылки на требования и ADR, структура репозитория, команды запуска и проверки. Проверить команды; для еще не реализованных частей указать планируемые проверки.
- **Сгенерировать проектные материалы.** По уточненному заданию получить и доработать с ИИ требования, пользовательские сценарии, диаграммы и черновые экраны из основной части. Сохранить редактируемые исходники; проверить согласованность ролей, сущностей, данных и бизнес-правил между ними.
- **Выбрать архитектуру.** Попросить ИИ предложить альтернативы минимум для двух существенных решений: например, хранения данных и разделения компонентов. Самостоятельно выбрать решения и оформить требуемые ADR с ограничениями и причинами отказа от альтернатив.
- **Подготовить приемку.** С помощью ИИ дополнить не менее трех пользовательских сценариев проверяемыми критериями успеха и ошибки. Связать каждый сценарий с требованием, сущностью и экраном; эти связи использовать при проектировании API в WebLab#2.
- **Провести проверку.** Поручить ИИ отдельный разбор противоречий в готовом комплекте. Зафиксировать минимум три собственных решения по результатам анализа: что уточнено, принято или отклонено и почему. Утвердить одну согласованную версию материалов для следующих лабораторных.

Результат и передача: Согласованный комплект проектных материалов, критерии сценариев, ADR и контекст агента → WebLab#2 и WebLab#7.

WebLab#2

REST API System Analyst

Depends on: WebLab#1

Subject: Проектирование REST API

Season: 2026

1. Спроектировать в формате OpenAPI 3.1 или более новой версии, поддерживаемой выбранными инструментами (<https://spec.openapis.org/oas/latest.html>), внешнее публичное API системы в идеологии REST, дающее доступ ко всем данным и функциям системы, необходимое для внешних интеграций и последующего подключения клиентского SPA-приложения. Пути (paths) в API должны иметь вид /api/v<номер версии>/...
2. Основным результатом выполнения ЛР является yaml файл с описанием API.
3. Если системе требуется взаимодействие, не выражаемое через REST, спроектировать и документировать отдельный контракт. Для однонаправленной потоковой доставки в клиент можно использовать SSE, для двунаправленного взаимодействия - WebSocket, для межсервисного RPC - gRPC, для асинхронной интеграции - брокер сообщений или webhook. Выбор механизма необходимо обосновать.
4. Предусмотреть полное покрытие CRUD хотя бы для одной из сущностей.
5. Предусмотреть хотя бы один пример частичного UPDATE.
6. Быть готовым обосновать и объяснить и выделение ресурсов, и перечень заложенных методов API в части как необходимости, так и достаточности для реализации задуманного функционала.
7. Быть готовым ответить на любые вопросы по всему семейству протоколов HTTP.
8. Проверить OpenAPI-спецификацию автоматическим линтером и проверить примеры запросов/ответов по схемам. Продемонстрировать mock-сервер либо воспроизводимую проверку схем и примеров без реализации бэкенда. Проверка реального API на соответствие контракту относится к WebLab#3.

Примечание 1: спроектировать именно **REST API** (а не просто HTTP/JSON/RPC API).

Руководствоваться <https://restfulapi.net/>.

Примечание 2: Во время проектирования API внимательно изучить методы HTTP и коды ответов, а также основные заголовки HTTP. Эта информация пригодится при сдаче лабораторной и на зачете.

Примечание 3: ЛР посвящена именно проектированию, а не реализации, чтобы не пришлось много раз переделывать 😊

ИИ-ТРЕК · WebLab#2

Обязательные требования для выбравших ИИ-трек.

Вход: Утвержденные требования, сценарии, сущности и архитектурные решения WebLab#1.

- **Спроектировать контракт с ИИ.** Поручить ИИ на основе материалов WebLab#1 выделить ресурсы и операции, затем сгенерировать OpenAPI в YAML. Включить схемы данных, стабильные operationId, параметры, примеры, применимые ошибки и правила доступа, если они предусмотрены проектом.
- **Проверить достаточность API.** С помощью ИИ составить таблицу «пользовательский сценарий → шаг → operationId» минимум для трех сценариев. Устранить недостающие операции и необоснованные методы. Проверить CRUD и частичное обновление из основного задания.

ИИ-ТРЕК · WebLab#2

Обязательные требования для выбравших ИИ-трек.

- **Проверить смысл контракта.** Поручить ИИ отдельное ревью методов HTTP, идемпотентности, кодов ответов, обязательных полей, пагинации и ошибок. Самостоятельно сверить спорные решения с требованиями и документацией; исправить спецификацию.
- **Выполнить автоматическую проверку.** С помощью агента настроить линтер OpenAPI и проверку примеров по схемам. Поднять mock-сервер по спецификации и пройти один многошаговый сценарий. Проверять контракт и примеры; реальная бизнес-логика появится в WebLab#3.
- **Управлять изменением.** Внести с ИИ одно содержательное уточнение, найденное при проверке: например, ограничение поля или дополнительный ответ об ошибке. Проверить diff YAML, повторить валидацию и актуализировать таблицу сценариев.
- **Передать результат дальше.** Зафиксировать проверенную версию OpenAPI и команды ее проверки. В этой лабораторной результат работы ИИ — спецификация и материалы проверки; сервер реализуется в WebLab#3, браузерный клиент генерируется в WebLab#8.

Результат и передача: Проверенная OpenAPI, таблица сценариев и воспроизводимые проверки контракта → WebLab#3; затем тот же актуальный контракт → WebLab#8.

Bonus #1

Спроектировать GraphQL API (дополнительно, как альтернативу REST).

WebLab#3

REST API Developer

Depends on: WebLab#2

Subject: Реализация REST API

Season: 2026

1. По спроектированному в формате OpenAPI REST API подготовить реализацию в программном коде.

ПРИМЕЧАНИЕ: для подготовки реализации API допускается классическая кодогенерация, использование кодирующего агента или специализированного сервиса (<https://openapi-generator.tech/>) на базе подготовленной API-спецификации. Независимо от способа реализации OpenAPI остается источником истины, а сгенерированный код подлежит проверке.

2. К реализации так же подключить swagger (для документирования). В деталях реализации допускаются вариации в зависимости от используемой технологии/языка.
3. Настроить отображение и полнофункциональную работу Swagger UI (проверка работоспособности API будет осуществляться с помощью него).
4. Придерживаться подходов “чистой архитектуры”: поддерживать абстрагирование СУБД путем использования паттерна Repository, использовать три модели сущности: сущность БД, сущность ядра (домена) и DTO для API. Для подключения слоя API вдохновляться семейством MV*-паттернов.
5. Имеющийся технологический (LR5 PPO) и графический (LR7 PPO, при наличии) интерфейсы необходимо поддерживать в рабочем состоянии. Это необходимо продемонстрировать одновременной работой TUI, GUI и API. При этом ожидается перенос TUI на работу с новым API. Перенос имеющегося GUI на новый API опционален (ниже вынесен в бонус).
6. Если в проекте уже есть api (в случае, если подключение GUI было реализовано посредством API), то оставить его в версии api/v1 и создать новое api/v2. Даже, если оно будет отличаться только исправлением ошибок следования методологии REST. Необходимо продемонстрировать работу обоих API.

Примечание: к приложению предъявляются все те же требования, что предъявлялись к приложениям в рамках курса ППО.

ИИ-ТРЕК · WebLab#3

Обязательные требования для выбравших ИИ-трек.

Вход: OpenAPI из WebLab#2, архитектура WebLab#1 и существующий код ППО.

- **Спланировать реализацию.** Поручить агенту сопоставить операции OpenAPI с существующим кодом и составить план изменений: API/DTO, домен, Repository, хранение данных, TUI и тесты. Утвердить границы слоев и совместимость ранее работающих интерфейсов.
- **Реализовать API с агентом.** Использовать агента для выполнения плана небольшими законченными частями. Сначала реализовать один сценарий от HTTP-запроса до данных и ответа, затем остальные операции основного задания. При необходимости настроить генератор серверного каркаса по OpenAPI.
- **Получить проверяемые тесты.** Поручить агенту создать тесты по спецификации и критериям WebLab#1: успешный запрос, неверные данные, отсутствующая сущность, частичное обновление; при наличии авторизации — отказ в доступе. Самостоятельно проверить ожидаемые результаты.

ИИ-ТРЕК · WebLab#3

Обязательные требования для выбравших ИИ-трек.

- **Подтвердить соответствие контракту.** Запустить тесты на реальном API и проверить коды ответов, схемы и обязательные поля. На одном примере показать, что тест обнаруживает намеренно внесенное локальное нарушение контракта; затем восстановить корректную реализацию и получить успешный прогон.
- **Выполнить ревью и исправления.** Проверить diff, статический анализ, разграничение DTO/домена/БД и обработку ошибок. Передать агенту воспроизводимые замечания и повторить проверки после исправлений. Требования и ожидаемые результаты тестов менять только при обоснованном изменении контракта.
- **Зафиксировать результат.** Сохранить работающий сервер, тесты, команды запуска и принятые изменения в репозитории. Swagger UI должен использовать согласованную спецификацию; при изменении требований сначала обновить OpenAPI и связанные сценарии, затем реализацию.

Результат и передача: Реализация API, тесты, действующая спецификация и команды запуска → WebLab#4.

Bonus #1

Реализовать простейшую аутентификацию и авторизацию. JWT допускается использовать при обоснованном выборе. Обращаться к защищенным методам API должен иметь возможность только авторизованный пользователь; публичные методы и правила доступа необходимо указать явно.

Bonus #2

Перенаправить GUI (легаси интерфейс) на новое REST API.

Bonus #3

Реализовать поддержку GraphQL API (по url /api/v2 или /api/v3).

WebLab#4

WEB SERVER Junior

Depends on: WebLab#3

Subject: WEB SERVER v.1.0

Season: 2026

1. Настроить асинхронный web-server для работы web-приложения в части маршрутизации.

ПРИМЕЧАНИЕ:

В качестве сервера можно использовать любой подходящий для задач маршрутизации и раздачи статики асинхронный сервер. Например: NGINX ([Руководство для начинающих](#)), Caddy, Traefik, envoy, haproxy, YARP.

2. Настроить маршрутизацию /api/v1 (/api/v2) на подготовленное REST API.
3. По пути /api/v1/docs (или /api/v2/docs для соответствующей версии) отдавать Swagger UI с описанием API. Маршрут документации не должен перекрывать маршруты операций API.
4. Если у системы был старый МРА или SPA интерфейс (PPO LR7) - проксировать его на /legacy. Если МРА-интерфейса не было, то сделать страничку /legacy с ссылкой на загрузку десктопной (PPO LR7)/консольной (PPO LR5) версии. Эта версия должна быть полнофункциональной и работать через API.
5. На путь /documentation настроить отдачу Readme.md с содержимым WebLab#1.
6. Настроить / на отдачу статики (в будущем - SPA-приложения). Пока положить приветственный HTML (/static/index.html) с картинками-эскизами будущего приложения (static/img/) из WebLab#1.
7. Настроить /reserved/ на отдачу той же страницы, что и /
8. Настроить /admin на проксирование в web-админку базы данных. После авторизации она должна открывать доступ к БД приложения; доступ к /admin необходимо ограничить и не публиковать в Интернет по умолчанию.
9. Настроить /status на отдачу страницы статуса сервера. Дополнительно предусмотреть /health и /ready либо эквивалентные проверки, различающие работоспособность процесса и готовность обслуживать запросы.
10. Сделать страницу /management, где с использованием простейшей HTML-верстки разместить ссылки на API и Swagger UI, /legacy, /documentation, /, /reserved/, /admin и /status. Ссылки на служебные страницы не должны обходить ограничения доступа.
11. Обеспечить воспроизводимый запуск web-сервера и приложения с помощью Docker Compose или эквивалентной декларативной конфигурации; команду запуска указать в README.md.

ИИ-ТРЕК · WebLab#4

Обязательные требования для выбравших ИИ-трек.

Вход: Работающее API WebLab#3, его спецификация, статические файлы и перечень маршрутов этой лабораторной.

- **Описать маршрутизацию.** Поручить ИИ составить таблицу «внешний путь → целевой компонент → префикс → ограничения доступа» для всех маршрутов основного задания. Самостоятельно проверить пересечения путей API, Swagger UI, статики и служебных страниц.

ИИ-ТРЕК · WebLab#4

Обязательные требования для выбравших ИИ-трек.

- **Сгенерировать конфигурацию.** С помощью агента подготовить конфигурацию web-сервера, Docker Compose или эквивалент, пример переменных окружения и инструкции запуска. Использовать реальные порты и зависимости проекта; секреты хранить отдельно от репозитория.
- **Сгенерировать проверки.** Поручить агенту создать запускаемый одной командой набор HTTP-проверок: маршруты, коды ответов, загрузка статики, сохранение префиксов, /health, /ready и ограничение доступа к /admin. Ожидаемые результаты сверить с утвержденной таблицей маршрутов.
- **Запустить и отладить.** Проверить синтаксис конфигурации, развернуть стенд и выполнить запросы. Если обнаружены ошибки, передать агенту фактические логи и ответы, проверить предложенное исправление и повторить запросы. Вывод ИИ о работоспособности без запуска не считается проверкой.
- **Проверить управляемость изменений.** Поручить агенту изменить внутренний порт одного компонента, сохранив внешние URL. Проверить diff, обновить конфигурацию и повторно пройти набор HTTP-проверок; зафиксировать рабочий итоговый вариант.
- **Подготовить передачу стенда.** Обновить README и контекст агента по итоговой конфигурации. Результат должен запускаться из чистого состояния по указанной команде; таблица маршрутов и проверки должны описывать именно этот стенд для WebLab#5 и WebLab#8.

Результат и передача: Конфигурация стенда, таблица маршрутов и HTTP-проверки → WebLab#5 и WebLab#8.

Bonus #1

Настроить визуализацию [Readme.md](#).

Bonus #2

Добавить визуальное оформление страницы status, используя только средства веб-сервера, подключаемые .css и .js файлы (цвета, шрифты, расположение, графика).

Bonus #3

Настроить количество worker-ов у сервера соответственно реальным возможностям системы, а также автостарт сервера при запуске системы.

WebLab#5

WEB SERVER Senior DevOps

Depends on: WebLab#4

Subject: WEB SERVER v.2.0

Season: 2026

1. Настроить web-server в части балансировки:
 - a. запустить 2 дополнительных инстанса бэкенда на других портах с правами доступа в базу данных только на чтение
 - b. настроить балансировку операций чтения (как правило, GET-запросов к /api/v1 или /api/v2) на 3 бэкенда в соотношении 2:1:1, где первый - основной экземпляр бэкенда;
 - c. операции, изменяющие состояние системы (как правило, POST, PUT, PATCH и DELETE), перенаправлять на один экземпляр с правами на запись;
 - d. Обработать ошибки при попытке обратиться на запись к инстансам на чтение
2. Провести нагрузочное тестирование с помощью k6, Locust, wrk, hey, ApacheBench или аналога. До теста сформулировать ожидаемое поведение и целевые показатели; в отчете .md указать конфигурацию, сценарий, объем нагрузки, полученные метрики и выводы.

Быть готовым показать и доказать, что балансировка действительно работает и почему она работает именно так.

3. Настроить web-server так, чтобы URL /mirror вел на отдельно развернутую версию приложения, а все относительные URL приложения корректно работали с новым префиксом (/mirror/api/v1 и др.). Настроить репликацию БД: multi-primary либо primary-replica в зависимости от возможностей СУБД. Продемонстрировать выбранный вариант, учесть задержку репликации и корректно обработать попытку записи в реплику только для чтения. Запросы на запись из /mirror направлять к узлу с правами на запись.

ВАЖНО: по путям / и /mirror должны полноценно работать полноценные версии приложений из п. 1

4. Настроить web server таким образом, чтобы подменялось имя сервера в заголовках http-ответов (проставлялось название приложения).
5. Настроить кэширование публичной статистики и gzip-сжатие подходящих текстовых ответов. API, персонализированные ответы и служебные страницы (/admin, /management, /status и проверки состояния) исключить из общего кэша. Правила кэширования проверить по заголовкам ответов.
6. Настроить по пути /monitoring визуализацию наблюдаемости: Grafana - для представления данных, Loki - для логов, Prometheus или аналог - для метрик. Данные должны поступать со всех экземпляров приложения.

ИИ-ТРЕК · WebLab#5

Обязательные требования для выбравших ИИ-трек.

Вход: Воспроизводимый стенд WebLab#4, сценарии API и целевые показатели WebLab#1.

- **Поставить эксперимент.** До запуска попросить ИИ предложить гипотезу об узком месте и план нагрузки: сценарии чтения/записи, данные, длительность, прогрев и критерии приемки. Утвердить допущения и ресурсные ограничения локального стенда.
- **Создать стенд с агентом.** Поручить агенту доработать конфигурации балансировки, репликации, кэширования и наблюдаемости по основному заданию. Проверить роли экземпляров, направление записи, маршруты / и /mirror и сбор данных со всех экземпляров.

ИИ-ТРЕК · WebLab#5

Обязательные требования для выбравших ИИ-трек.

- **Подготовить нагрузку и сбор метрик.** С помощью агента подготовить воспроизводимый сценарий нагрузочного теста и необходимые запросы/панели наблюдаемости. Измерить пропускную способность, р95 задержки, долю ошибок и фактическое распределение чтений 2:1:1; сохранить исходные результаты.
- **Проанализировать и улучшить.** Передать ИИ измерения и конфигурацию, сопоставить выводы с первоначальной гипотезой. Выбрать одно обоснованное изменение, выполнить его с агентом и повторить тест при сопоставимых условиях. Объяснить улучшение, отсутствие эффекта или ухудшение; сохранить лучший обоснованный вариант.
- **Проверить отказ.** С помощью ИИ подготовить и выполнить на локальном стенде один сценарий недоступности экземпляра. Проверить обнаружение отказа, ответы клиенту, восстановление и отсутствие перенаправления записи на экземпляр только для чтения.
- **Оформить инженерный вывод.** С помощью ИИ собрать отчет из реальных данных: конфигурация, команды, условия, результаты до/после и выводы. Самостоятельно сверить цифры и графики с исходными измерениями; указать, какие предположения ИИ подтвердились и какие остались непроверенными.

Результат и передача: Работающий стенд, сценарии нагрузки и отчет с измерениями → WebLab#6.

Bonus #1

Настроить HTTPS (<https://letsencrypt.org/docs/certificates-for-localhost/>) и HTTP/2 для всех запросов. На странице index.html и изображении продемонстрировать preload либо 103 Early Hints.

Bonus #2

Настроить работу сервиса по протоколу http3.

Bonus #3

Настроить активные HTTP-проверки доступности upstream-серверов средствами выбранного web-сервера или эквивалентного компонента. Показать исключение недоступного экземпляра из балансировки и возврат после восстановления. Использовать проверки /health и /ready из WebLab#4; само наличие этих URL бонусом не считается.

Bonus #4

Настроить дополнительно кластерный режим СУБД ([для PostgreSQL](#))

Bonus #5

Настроить Service Discovery ([пример с Consul](#))

WebLab#6

Backender

Depends on: WebLab#5

Subject: SOA

Season: 2026

1. Декомпозировать бэкенд приложения как минимум на 3 независимо развертываемых сервиса. Data, Core (Domain) и Gateway могут оставаться логическими слоями, но сами по себе не являются достаточным обоснованием границ сервисов. Границы должны следовать бизнес-возможностям или независимым обязанностям; контракты между сервисами необходимо определить явно.
2. Для каждого существенного взаимодействия между сервисами выбрать подходящий механизм и обосновать выбор:
 - a. Синхронный
 - i. REST API
 - ii. gRPC
 - iii. GraphQL
 - iv. SOAP
 - b. Асинхронное взаимодействие:
 - i. Брокеры сообщений (RabbitMQ/Kafka)
 - ii. Webhooks
 - c. Двухнаправленное взаимодействие в реальном времени: WebSocket (если оно действительно необходимо).

Для асинхронных сценариев рекомендуется механизм на основе брокеров сообщений (RabbitMQ/Kafka). Выбор между синхронным вызовом и асинхронной передачей определяется требованиями конкретного взаимодействия.

3. Добавить логи и распределенные трассировки новых сервисов в /monitoring. Использовать OpenTelemetry или эквивалент и обеспечить единый идентификатор корреляции запроса.
4. Добиться корректной работы с учетом репликации и зеркалирования из WebLab#5. Каждый выделенный сервис должен запускаться в трех экземплярах и корректно взаимодействовать с остальными. Экземпляры сервиса и реплики БД различать: правила чтения и записи применять в соответствии с ответственностью сервиса и выбранной схемой хранения данных.
5. Предусмотреть тайм-ауты, ограниченные повторные попытки и идемпотентность для операций, которые могут выполняться повторно; продемонстрировать обработку временной недоступности одного сервиса.

ИИ-ТРЕК · WebLab#6

Обязательные требования для выбравших ИИ-трек.

Вход: Код WebLab#3, инфраструктура и измерения WebLab#5, действующие внешние контракты.

- **Выбрать границы сервисов.** Поручить ИИ предложить два варианта декомпозиции с учетом бизнес-возможностей, владения данными и измерений WebLab#5. В ADR сравнить связность, сложность эксплуатации и стоимость межсервисных вызовов; выбрать вариант минимум с тремя сервисами по основному заданию.

ИИ-ТРЕК · WebLab#6

Обязательные требования для выбравших ИИ-трек.

- **Спроектировать переход.** С помощью агента составить поэтапный план выделения сервисов, их контракты и диаграмму одного сквозного сценария. Указать владельцев данных и сохранить совместимость внешнего API WebLab#2; оценку модульного монолита можно добавить как альтернативу в ADR.
- **Выполнить рефакторинг с агентом.** Поручать агенту выделение сервисов по частям: код, адаптеры взаимодействия, конфигурация запуска и тесты. После каждого шага проверять существующий пользовательский сценарий; в итоге развернуть требуемые сервисы и их экземпляры.
- **Проверить распределенное поведение.** Сгенерировать с агентом контрактные и интеграционные проверки взаимодействий. Для выбранного сценария указать тайм-ауты, допустимость повторов, защиту от повторного эффекта и ожидаемое поведение при недоступности зависимого сервиса.
- **Получить доказательства из системы.** Поручить агенту добавить требуемые логи и трассировки. Выполнить локально один сценарий отказа, проследить запрос через сервисы и сопоставить реальное поведение с диаграммой. Ошибки устранять по наблюдаемым данным с повторным прогоном проверок.
- **Завершить миграцию.** С помощью ИИ актуализировать диаграммы, ADR, контракты и команды запуска по фактическому устройству системы. В AI_REVIEW.md показать одно решение о границах сервисов и один проверенный компромисс надежности или производительности.

Результат и передача: Сервисы, контракты, проверки отказа, трассировки и актуальные архитектурные материалы.

Bonus #1

Разработать CI/CD для приложения со сборкой docker-образов, запуском тестов, доставкой и развертыванием.

Bonus #2

Выделить сервис отдельной авторизации (для реализации SSO-like подхода).

Bonus #3

Настроить расширенную трассировку OpenTelemetry: осмысленные атрибуты, sampling, связь трассировок с логами и визуализацию межсервисного сценария.

WebLab#7

Designer

Depends on: WebLab#1

Subject: Макет

Season: 2026

Необходимо для выбранного приложения разработать макет пользовательского интерфейса в Figma, Pixso или другом аналоге, ссылку на макет разместить в Readme.md репозитория. Для каждой из трех стадий работы создать отдельную страницу проекта в Figma/Pixso/другом аналоге.

Исследование

1. Перед разработкой самого макета необходимо проработать портреты пользователей и соответствующие пользовательские сценарии.
 - a. Если предполагается общедоступное публичное приложение — на этом этапе четко позиционировать ЦА и выбрать из нее 2-3 ключевых пользовательских портрета (Customer Profile).
 - b. Если приложение проектируется для конкретных пользователей (бизнес-приложение), то также описать портрет таких пользователей, при необходимости сделать уклон в разделение по ролям/правам доступа.
 - c. Для каждого портрета, в соответствии с подготовленными в WebLab#1 UseCase-диаграммами составить пользовательские сценарии.
2. Собрать доску настроения (moodboard— см. приложения) минимум из 15 изображений. Можно сделать это на отдельной странице в Figma/Pixso, а можно на [Pinterest](https://www.pinterest.com) в виде отдельной доски. Ссылку на неё в таком случае добавить в Readme.md.

Стили

3. Опираясь на портреты пользователей, подготовленные сценарии и доску настроения разработать дизайн-язык — краткое описание общей стилистики/дизайна, включая:
 - a. Цветовую палитру в HEX-кодах (1 основной цвет, 1-3 дополнительных/акцентных, а также цвета для состояний «успех», «ошибка», «предупреждение», «информационное сообщение»). Для пар текста и фона проверить достаточный контраст. Для вдохновения можно обратиться к сервисам вида <http://coolors.co/>.
 - b. Типографику: шрифты (не более 2) и их размеры (всего не более 8 различных начертаний на весь проект)
 - c. Размеры - лучше использовать один базовый размер, а остальные делать дольными или кратными (пример: 16px основной размер, дольные - 4px, 8px, кратные - 32px, 48px, 64px)
 - d. Примеры геометрии (радиусы скруглений, острые или тупые углы, геометрические формы)
 - e. Примеры иконок и иллюстраций
 - f. Если применимо для вашего проекта, укажите точные настройки для теней, свечений, размытий и прочих ультрамодных эффектов.
 - g. Необходимо не просто накидать произвольных цветов и начертаний на холст, а указать их назначение за счёт подписей (к примеру, #B5ACAA — background color, #B5ACAA — цвет текста)

Создание макета

4. Основываясь на разработанных сценариях, подготовить макет Figma/Pixso, в макете должны быть проработаны все необходимые экраны приложения.
5. При разработке макета строго придерживаться следующих требований:
 - a. Использовать глобальные стили и/или design tokens для всех переиспользуемых цветов, теней, свечений, начертаний шрифтов, размеров и других системных параметров.
 - b. Использовать сетку для размещения компонентов, не должно быть висящих в случайных местах элементов (за редким исключением, когда это элемент дизайна, такое нужно согласовать с преподавателем).
 - c. Использовать компонентный подход — все переиспользуемые элементы интерфейса должны быть организованы в виде компонентов. Иногда также полезно выносить в виде отдельного логически завершенного компонента не переиспользуемые элементы. Все мастер-компоненты упорядочить и подписать на отдельной странице проекта.
 - d. Строго соблюдать иерархию в списке слоёв. Следить за тем, чтобы визуальная вложенность элементов друг в друга на холсте также была отражена в логической организации иерархии слоёв в их списке слева.
 - e. Для каждого компонента продумывать модель адаптивности на основе привязок (constraints). Компоненты и экраны не должны ломаться при масштабировании (разумеется, в определённых пределах, которые вы должны определить сами)
 - f. Семантически именовать фреймы (экраны), компоненты и группы. Низкоуровневые вспомогательные группы внутри компонентов можно оставлять неименованными.
 - g. **Не использовать** структурное именование (пример: TopLeftBar, RightMiddleIcon, SecondTab и т.д. Оно попросту перестаёт работать при добавлении адаптивности)
 - h. Каждый экран приложения (или состояния одного экрана) размещается в отдельном Frame, название которого отражает смысловую роль экрана в приложении и его текущее состояние.
 - i. При сочетании скругления и обводки проверить равномерность видимой толщины границы и корректность сопряжения внутреннего и внешнего радиусов.
 - j. Предусмотреть применимые состояния загрузки, отсутствия данных, ошибки, успешного выполнения и недоступности элемента управления.
 - k. На уровне макета проверить контраст, предусмотреть видимый фокус и текстовые подписи, описать порядок фокуса и клавиатурное поведение ключевого сценария. Проверка работающего интерфейса с клавиатуры выполняется в WebLab#8.

ИИ-ТРЕК · WebLab#7

Обязательные требования для выбравших ИИ-трек.

Вход: Пользовательские сценарии, роли, ограничения и черновые экраны WebLab#1.

- **Сформировать дизайн-бриф.** Поручить ИИ превратить сценарии и ограничения WebLab#1 в бриф: пользователи, задачи, контекст использования и критерии удобства. Сгенерированные портреты и предположения пометать как гипотезы; результаты реального исследования указывать отдельно.
- **Исследовать варианты.** С помощью ИИ подготовить 2–3 визуальных направления или варианта компоновки ключевого сценария. Сопоставить их с брифом, выбрать направление и объяснить принятые и отклоненные решения. Собрать moodboard по требованиям основной части.

ИИ-ТРЕК · WebLab#7

Обязательные требования для выбравших ИИ-трек.

- **Создать дизайн-систему.** Использовать ИИ для подготовки палитры, типографики, design tokens и перечня компонентов с состояниями. Перенести принятые решения в редактируемые стили и компоненты выбранного инструмента; проверить единообразие и назначение токенов.
- **Сгенерировать и доработать экраны.** Получить с помощью ИИ черновики экранов ключевых сценариев, затем довести макет до требований задания. Сохранить структуру исследования, стилей и макета в Figma/Pixso или аналоге; при отсутствии интеграции с ИИ перенести результаты вручную.
- **Провести ревью макета.** Передать ИИ экраны и критерии сценариев, получить замечания к полноте действий, понятности текста, состояниям и адаптивности. Самостоятельно проверить контраст и описать порядок фокуса, подписи и клавиатурное поведение. Исправить принятые замечания в макете.
- **Подготовить передачу в разработку.** С помощью ИИ собрать описание компонентов, токенов, состояний и требуемых экрану данных для WebLab#8. Указать переходы между экранами; подключение к реальному API и проверка клавиатуры в работающем интерфейсе выполняются в WebLab#8. Интерактивный прототип остается бонусом.

Результат и передача: Редактируемый макет, компоненты, токены и описание состояний → WebLab#8.

Bonus #1

Настроить навигацию между экранами в Figma/Pixso (prototype). Так приложение можно будет потыкать в режиме просмотра.

Bonus #2

Упаковать дизайн язык и макеты в стилевой планшет (stylescape — см. приложения). Реализовать его на отдельной странице в Figma, переиспользовать на нём экраны и/или компоненты.

Bonus #3

Собрать полностью интерактивный макет, добавить анимацию (не менее 3-х анимированных элементов).

Приложения**Доска настроения (moodboard)**

Доска настроения должна передавать настроение в предметной области вашего приложения. Что должен чувствовать ваш пользователь — веселье, удовольствие, летнее тепло или суровую непоколебимость, надёжность, футуристичность. Ваша задача передать это настроение, собрав подходящие картинки. На базе доски настроения обычно формируется палитра.

Стилевой планшет (stylescape)

На стилевом планшете изображена палитра, начертания шрифтов, показана геометрия и примеры элементов управления (controls), передана атмосфера посредством иллюстраций и фотографий, приведены примеры иконок. Всё это обычно увенчивает логотип.

WebLab#8

Frontender

Depends on: WebLab#4 & WebLab#7

Subject: SPA => Remote Facade + Business Logic + UI

Season: 2026

Разработать пользовательский интерфейс в виде SPA, выделив слои взаимодействия с API, бизнес-логики и представления. Допускается гибридный фреймворк с SSR/SSG при сохранении клиентской маршрутизации и интерактивности в режиме SPA.

1. Приложение должно быть разработано на языке TypeScript с использованием JS-фреймворка, обеспечивающим компонентный подход (например, Angular, React, Vue и др.), либо без фреймворка, но с теми же требованиями.
2. Явно разделить представление, состояние/бизнес-логику и взаимодействие с API. Использовать MVC, MVP, MVVM или другой обоснованный подход. Не допускаются неконтролируемые подписки и ручные модификации DOM, в том числе на основе jQuery.
3. Запрещается использование alert(), confirm(), prompt().
4. Реализация бизнес-логики приложения, API должна быть вынесена в соответствующие программные компоненты (обычно именуемые “сервисами”).
5. Весь UI должен быть действительно декомпозирован на компоненты, все повторяющиеся и логически выделяемые элементы интерфейса должны быть вынесены в качестве иерархической системы ui-компонентов (панели, кнопки, поля, формы ввода, списки, экраны).
6. Открываемую страницу и параметры для передачи по ссылке (фильтр, сортировку, номер страницы) кодировать в URL и восстанавливать по прямой ссылке. Черновики форм и секретные данные хранить в состоянии приложения или подходящем хранилище, за пределами URL.
7. Для вызова API использовать fetch(), axios или другой браузерный HTTP-клиент. Клиент можно генерировать из актуальной OpenAPI WebLab#2; команду сохранить в проекте. Адреса и типы данных должны соответствовать контракту.
8. По умолчанию - достаточно десктопной версии.
9. В package.json прописать сценарий сборки (секция “scripts” и использовать в сценарии сборщик типа vite <https://vite.dev/>). Если ранее реализован CI, добавить сборку в него.
10. Компонентные и интеграционные тесты должны проверять ключевую логику и критические пользовательские сценарии. Покрытие — не менее 60% строк кода приложения, включая код ИИ; зависимости и генерируемый SDK API исключить из расчета. Процент не заменяет содержательных сценариев. Рекомендуются Jest/Vitest или аналоги; при наличии CI запускать в нем тесты.
11. Проверить ключевые страницы на семантическую разметку, управление с клавиатуры и контраст; зафиксировать результаты проверки производительности с помощью Lighthouse или аналога.

ИИ-ТРЕК · WebLab#8

Обязательные требования для выбравших ИИ-трек.

Вход: Макеты WebLab#7, актуальная OpenAPI WebLab#2 и API, доступное через web-сервер WebLab#4.

- **Спроектировать реализацию с агентом.** Передать макеты, токены, OpenAPI и правила проекта; получить план компонентов, маршрутов, состояния и слоя API. Сопоставить действия пользователя с operationId. Если контракту не хватает данных, сначала согласовать изменение спецификации и сервера.

ИИ-ТРЕК · WebLab#8

Обязательные требования для выбравших ИИ-трек.

- **Настроить генерацию клиента.** С помощью агента настроить воспроизводимую генерацию типизированного клиента из утвержденной OpenAPI, например штатным генератором. Сохранить команду и версию инструмента; бизнес-логику и UI разместить отдельно от регенерируемого кода.
- **Реализовать интерфейс с агентом.** Создавать компоненты и сценарии последовательными изменениями: сначала один рабочий путь «экран → API → результат», затем остальные экраны задания. Использовать токены макета, учитывать загрузку, пустые данные, ошибки и сохранение состояния маршрута.
- **Сгенерировать и проверить тесты.** Поручить агенту создать компонентные и интеграционные тесты по пользовательским сценариям: успешное действие, валидация и ошибка API. Проверить ожидаемые результаты, выполнить тесты, проверку типов и сборку; сохранить требуемый отчет о покрытии.
- **Принять результат в браузере.** Сопоставить реальные экраны с макетами, проверить прямую ссылку, обновление страницы, клавиатуру и состояния интерфейса. Для ревью с ИИ использовать доступ к браузеру либо реальные скриншоты и наблюдения. Исправить подтвержденные замечания; E2E-автоматизация остается бонусом.
- **Провести изменение по задаче.** С помощью агента выполнить одно небольшое изменение, например добавить фильтр или состояние ошибки. Проверить затронутые компоненты, diff и тесты; при изменении API обновить контракт и регенерировать клиент. Зафиксировать результат в AI_REVIEW.md.

Результат и передача: Работающий интерфейс, генерация API-клиента и проверенные сценарии → WebLab#9.

Bonus #1

Поддержать аутентификацию и авторизацию, реализованные на бэкенде в WebLab#3: например, сессию с cookies или JWT. Продемонстрировать вход, выход, истечение сессии/токена и отображение отказа в доступе.

Bonus #2

Подключен и настроен и зарегистрирован как precommit hook линтер, а также добавлен запуск линтера в CI (если CI был реализован ранее). Пример инструмента для работы с precommit hooks: <https://www.npmjs.com/package/lefthook>.

Bonus #3

Адаптив на десктоп (корректное отображение в диапазоне как минимум трех разрешений).

Bonus #4

Мобильная версия.

Bonus #5

E2E-тесты с использованием фреймворка Playwright или аналога на основные сценарии приложения, а также их прогон в CI (если последний реализован).

Bonus #6

Использовать поддерживаемую архитектуру стилей: нативный CSS с custom properties, nesting/layers либо SASS/другой препроцессор. Выбор необходимо обосновать, а преобразование стилей включить в сборку.

WebLab#9

Frontender PRO MAX

Depends on: WebLab#8

Subject: SPA: Extra

Season: 2026

1. Упаковать приложение в TG Web App (продемонстрировать одновременную работу всех ранее разработанных UI).
2. Обеспечить сохранение своего состояния (частичная автономность), которое сохраняется во внутреннем хранилище (localStorage, IndexedDB). Например, сохраняется частично заполненная, но не отправленная форма и т.д.
3. Добавить WebAssembly (WASM). В WASM вынести реальный вычислительный алгоритм и обосновать, почему для него подходит эта технология; пустая обертка или формальный вызов функции не засчитываются.
4. Вынести сложную обработку данных в Web Worker.

ИИ-ТРЕК · WebLab#9

Обязательные требования для выбравших ИИ-трек.

Вход: Работающее приложение WebLab#8, его архитектура и выбранный вычислительный сценарий.

- **Спланировать расширения.** Поручить ИИ предложить план реализации четырех основных пунктов: Telegram Web App, сохранение состояния, WASM и Web Worker. Для вычислительного сценария выбрать границы модулей и данные для сравнения; WASM допускается выполнять внутри Worker.
- **Реализовать модули с агентом.** С помощью агента создать адаптер Telegram, слой локального хранения, вычислительный модуль WASM и взаимодействие с Worker. Проверить типы сообщений, передачу данных, обработку ошибок и жизненный цикл Worker; сохранить работу прежних интерфейсов.
- **Проверить интеграции.** Сгенерировать с агентом проверки восстановления незавершенного действия после перезагрузки и ошибок фоновой обработки. Продемонстрировать сценарий в браузере и Telegram. Если данные Telegram используются для входа, проверять initData на сервере; секреты бота в клиент не включать.
- **Измерить эффект.** Попросить агента подготовить сравнение обычной JS-реализации в основном потоке и выбранного решения с WASM/Worker на одинаковых данных. Проверить правильность результата, время вычислений и отзывчивость UI с учетом передачи данных и запуска модулей.
- **Принять архитектурное решение.** Передать ИИ результаты измерений, проверить его объяснение и при необходимости доработать решение. В ADR обосновать границы модулей и ограничения. Отсутствие ускорения допустимо при корректном эксперименте и содержательном выводе; сами требования к WASM и Worker сохраняются.
- **Проверить управляемость проекта.** В отдельной новой сессии дать агенту актуальный контекст репозитория и небольшую задачу по одному расширению. Проверить план, diff и результат запуска. Дополнить инструкции недостающими сведениями; в AI_REVIEW.md показать, что обеспечило воспроизводимость работы.

Результат и передача: Расширенное приложение, проверки, сравнение измерений и актуальные инструкции агенту.

ИИ-ТРЕК · WebLab#9

Обязательные требования для выбравших ИИ-трек.

По желанию: ИИ-функция продукта. Добавить одну ограниченную функцию по предметной области: например, классификацию обращения или подготовку сводки. Это добровольное расширение ИИ-трека; оно не заменяет WASM/Worker и не меняет существующие бонусы.

Архитектура и приемка расширения. Внешнюю модель вызывать через backend/BFF с серверным хранением ключей. Определить схему результата, валидацию, тайм-ауты и лимит затрат; при вызове инструментов разрешить только нужные операции и проверять права на сервере. На 15–20 сценариях, включая ошибки и prompt injection, оценить качество и задержку. Для локальной модели в браузере описать ограничения устройства. Материалы WebLab#6 для этого расширения не обязательны.

Bonus #1

Провести проверку совместимости в актуальных стабильных версиях Chrome, Firefox, Safari и Edge либо в согласованном наборе браузеров. Для одной неподдерживаемой возможности продемонстрировать graceful degradation, fallback или polyfill.

Bonus #2

Использовать отдельный инструмент управления состоянием (например, Effector, Redux Toolkit, Pinia, Zustand или аналог) и обосновать его необходимость для проекта.

Bonus #3

Добавить Canvas с небольшой анимированной пасхалкой (мини-игра, заставка с звездным небом или фигурой, отталкивающейся от границ экрана).

Bonus #4

Сделать сборку Frontend-приложения не менее чем в одном из вариантов (бонус за каждый из вариантов):

- в Desktop с использованием Electron
- в мобильное приложение с использованием технологии WebView и инструментов для интеграции JavaScript и нативного кода вида Capacitor;
- в расширение для Chromium-based браузера.

Bonus #5

Встроить в процесс сборки приложения компиляцию программы на C/C++/Rust или другом подходящем языке в WASM.

Bonus #6

Сделать приложение автономным в браузере для его основных предметных сценариев: заменить обращение к API локальным адаптером хранения, например на IndexedDB. Для повторного открытия без сети после первого успешного запуска использовать Service Worker и Cache API. Доступ к IndexedDB можно вынести в Web Worker; localStorage доступен в контексте окна, а не внутри Worker. Продемонстрировать повторное открытие приложения, чтение и изменение локальных данных без сети. Внешние интеграции обозначить как недоступные в этом режиме.